

Cluster Rush - Test-Driven Development Plan (REVISED v2)

Status: RETROACTIVE TDD APPROACH - Existing code will be tested first, then refactored. No deletion of working code. **Current Coverage:** ~0% (only 1 test file with 6 passing tests) **Target Coverage:** 90%+ overall, 100% for critical paths **Timeline:** 4 Weeks (adjusted from 3 weeks for realistic scope)

TDD Philosophy (STRICT ENFORCEMENT)

NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST Every implementation follows strict RED → GREEN → REFACTOR cycle.

Mandatory TDD Steps (Never Skip)

1. **RED:** Write failing test describing expected behavior
2. **VERIFY RED:** Run test to confirm failure (not error)
3. **GREEN:** Write minimal code to pass test
4. **VERIFY GREEN:** Run test to confirm pass
5. **REFACTOR:** Clean code without changing behavior
6. **VERIFY REFACTOR:** All tests still pass

TDD Violations (Task Requires Review)

- Code written before test → Write test first, then refactor
- Test passes immediately → Fix test to fail first
- "I'll test later" attitude → Stop, follow TDD
- Modifying existing code without test → Write test first

Retroactive TDD Strategy (For Existing Code)

1. **Test Existing Behavior First:** Write tests that pass against current code
2. **Identify Gaps:** Find untested code paths
3. **Add Failing Tests:** Write tests for missing functionality

4. **Refactor Incrementally:** Improve code while maintaining test pass
 5. **Never Delete Working Code:** Only add tests, then improve
-

COMPLETE FEATURE SPECIFICATION

Core Gameplay Mechanics

- **Player Movement:** WASD/Arrow keys, smooth acceleration/deceleration
- **Jump Mechanics:** SPACE key, grounded state detection, double-jump capability
- **Gravity System:** Constant downward acceleration, terminal velocity
- **Collision Detection:** Player-obstacle, player-powerup, player-environment
- **Score System:** Distance-based scoring, multipliers, high score persistence
- **Game States:** Menu → Playing → Paused → Game Over → Restart

Game Modes

1. **Endless Mode:** Infinite procedurally generated course (MVP - Priority 1)
2. **Time Attack:** Complete as much as possible in 60 seconds (Phase 2)
3. **Challenge Mode:** Specific obstacle patterns with objectives (Phase 2)

Obstacle Types (5 types)

1. **Static Blocks:** Fixed position, simple collision
2. **Spikes:** Instant death on contact
3. **Moving Obstacles:** Horizontal/vertical movement patterns
4. **Rotating Obstacles:** Angular movement, timing-based avoidance
5. **Breakable Obstacles:** Can be destroyed with power-ups

Power-Up Types (4 types)

1. **Shield:** Temporary invincibility (5 seconds)
2. **Speed Boost:** Increased movement speed (3 seconds)
3. **Magnet:** Attracts nearby score items (5 seconds)
4. **Score Multiplier:** 2x score for 10 seconds

UI Components

- Main Menu (Start, Settings, High Scores)
- HUD (Score, Health, Power-up indicators)
- Pause Menu (Resume, Restart, Quit)
- Game Over Screen (Final score, High score comparison, Restart option)
- Settings (Audio levels, Controls, Graphics options)

Audio System

- Background music (looping tracks per scene)
 - Sound effects (jump, collision, power-up collection, UI interactions)
 - Audio mixing and volume controls
-

WEEK 0: INFRASTRUCTURE & BASELINE (NEW)

Day 0.1: Testing Infrastructure Setup

Task 0.1.1: Vitest Configuration - RED: Test that `npm run test` fails (no config) - GREEN: Configure Vitest with TypeScript, browser environment, coverage - VERIFY: `npm test` → "✓ 0 passed" (empty but running) - Acceptance Criteria: - [] Vitest configured in `vite.config.ts` - [] Test command runs without errors - [] Coverage reporting enabled (c8 or vitest coverage) - [] Browser environment configured (happy-dom or jsdom)

Task 0.1.2: Mocking Strategy - RED: Test that Three.js/Cannon-es mocks work - GREEN: Create `tests/mocks/` directory with: - `mockThree.js` - Stub Three.js classes - `mockCannon.js` - Stub Physics classes - `mockAudio.js` - Stub AudioContext - VERIFY: Unit tests run without actual WebGL context - Acceptance Criteria: - [] Three.js classes can be mocked - [] Cannon-es physics can be stubbed - [] Audio system can be mocked - [] Mocks return predictable values

Task 0.1.3: CI/CD Pipeline - RED: Test that GitHub Actions workflow fails (no workflow) - GREEN: Create `.github/workflows/test.yml` with: - Run tests on push to main - Enforce coverage threshold (80% minimum) - Block merge if tests fail - VERIFY: Push triggers CI, tests run, status check updates - Acceptance Criteria: - [] GitHub Actions workflow exists - [] Tests run

automatically on push - ☐ Coverage threshold enforced - ☐ Build status visible in PR

Task 0.1.4: Pre-commit Hooks - RED: Test that commit without tests is blocked - GREEN: Setup Husky + lint-staged to: - Run tests before commit - Block commit if tests fail - Show coverage report - VERIFY: `git commit` runs tests, blocks on failure - Acceptance Criteria: - ☐ Husky installed - ☐ Pre-commit hook runs tests - ☐ Commit blocked if tests fail

Day 0.2: Existing Code Baseline

Task 0.2.1: Code Inventory - RED: Test that all existing source files are documented - GREEN: Create `tests/baseline/INVENTORY.md` listing: - All source files in `src/` - All existing tests in `tests/` - Coverage gaps identified - VERIFY: Inventory matches actual file structure - Acceptance Criteria: - ☐ All source files cataloged - ☐ Existing tests identified - ☐ Coverage gaps documented

Task 0.2.2: Retroactive Tests for Existing Code - RED: Test that existing Game class has no tests - GREEN: Write tests for existing `Game.ts`, `Player.ts`, `Input.ts` - VERIFY: Tests pass against existing implementation - Acceptance Criteria: - ☐ All existing public methods have tests - ☐ Tests verify current behavior (not desired behavior) - ☐ Baseline coverage established (~20% target)

WEEK 1: CORE INFRASTRUCTURE (TDD VERIFIED)

Day 1: Project Setup with Tests First

Task 1.1: Initialize project with testing infrastructure - RED: Test that `npm run test` runs empty test suite - GREEN: Configure Vitest with TypeScript support and browser environment - VERIFY: `npm test` → "✓ 0 passed" (empty but running) - Acceptance Criteria: - ☐ Vitest configured in `vite.config.ts` - ☐ Test command runs without errors - ☐ Empty test suite passes

Task 1.2: TypeScript strict mode verification - RED: Test that TypeScript compilation fails with current strict settings - GREEN: Configure `tsconfig.json` with strict options - VERIFY: `npm run build` compiles without type errors - Acceptance Criteria: - ☐ `strict: true` in `tsconfig` - ☐ No TypeScript errors on build - ☐ Type checking enforced in CI

Task 1.3: WebGL renderer test (with mock) - RED: Test that WebGL renderer creation fails (no implementation) - GREEN: Create minimal Three.js renderer in `core/Renderer.ts` with mock support - VERIFY: Test passes, renderer created successfully (mocked in unit tests) - Acceptance Criteria: - ☐ Renderer creates WebGL context - ☐ Renderer has correct dimensions - ☐ Renderer clears screen properly - ☐ Mock renderer works in unit tests

Day 2: Core Game Loop (Test-Driven)

Task 2.1: Game state management - RED: Test that Game class exists and has correct initial state - GREEN: Implement Game class with start/stop/pause methods - VERIFY: Game starts in STOPPED state, transitions correctly - Acceptance Criteria: - ☐ Game class has all required states - ☐ State transitions are valid (no illegal transitions) - ☐ Game emits state change events - ☐ **Error Handling:** Invalid transitions throw descriptive errors

Task 2.2: Frame loop timing - RED: Test that game loop calls update with `deltaTime` - GREEN: Implement `requestAnimationFrame` loop in `GameLoop.ts` - VERIFY: Update called ~60 times per second with accurate `deltaTime` - Acceptance Criteria: - ☐ Delta time accuracy within 5% - ☐ Frame rate target of 60 FPS - ☐ Loop stops when game stops - ☐ **Edge Case:** Handles frame drops gracefully (step > 100ms)

Task 2.3: Scene management system - RED: Test scene creation and object addition fails - GREEN: Scene class with Three.js integration in `SceneManager.ts` - VERIFY: Objects appear in scene when added - Acceptance Criteria: - ☐ Scenes can be registered/unregistered - ☐ Scene transitions work correctly - ☐ Active scene receives update calls - ☐ **Error Handling:** Missing scene throws descriptive error

Day 3: Input System Testing

Task 3.1: Keyboard input detection - RED: Test keydown events trigger input state changes - GREEN: DOM event listener with key mapping in `Input.ts` - VERIFY: WASD keys detected in automated browser test - Acceptance Criteria: - ☐ All keyboard keys mapped correctly - ☐ Key state persists while pressed - ☐ Key release detected properly - ☐ **Edge Case:** Key repeat handled correctly

Task 3.2: Input buffering for frame independence - RED: Test input state persists across frames - GREEN: Input buffer with frame processing - VERIFY: Input works at high and low frame rates - Acceptance Criteria: - ☐ Input not affected by frame rate variations - ☐ Input queue processes correctly - ☐ No input lag detected - ☐ **Edge Case:** Input lost on window blur handled

Day 4: Asset Loading System (MOVED FROM WEEK 3)

Task 4.1: Asset loader creation - RED: Test asset loading fails (no implementation) - GREEN: `AssetLoader.ts` with async loading, caching, error handling - VERIFY: Assets load successfully, cache works - Acceptance Criteria: - ☐ GLTF models load correctly - ☐ Textures load with fallback - ☐ Audio files load with format detection - ☐ **Error Handling:** 404 returns fallback asset, not crash

Task 4.2: Loading progress tracking - RED: Test progress events fire during load - GREEN: Progress events with percentage tracking - VERIFY: Progress bar updates during asset load - Acceptance Criteria: - ☐ Progress events fire at correct intervals - ☐ 100% fires when all assets loaded - ☐ Error events fire on failure

WEEK 2: GAMEPLAY SYSTEMS

Day 5: Player Entity TDD

Cycle 1: Player creation - RED: Test player mesh created with proper geometry - GREEN: Box geometry with material in `Player.ts` - VERIFY: Player appears at starting position - Acceptance Criteria: - ☐ Player mesh has correct geometry - ☐ Player positioned at spawn point - ☐ Player has proper material

Cycle 2: Movement - RED: Test player position changes with velocity - GREEN: Apply velocity based on input - VERIFY: WASD moves player in corresponding directions - Acceptance Criteria: - ☐ Player moves in all 4 directions - ☐ Movement speed is consistent - ☐ Movement stops when input released - ☐ **Edge Case:** Movement clamped to world bounds

Cycle 3: Jump mechanics - RED: Test player jumps with SPACE when grounded - GREEN: Apply upward impulse, track ground state - VERIFY: Player ascends then falls back - Acceptance Criteria: - ☐ Jump only works when grounded - ☐ Jump height is consistent - ☐ Gravity pulls player down after jump - ☐ **Edge Case:** Double-jump limited to configured count

Cycle 4: Gravity application - RED: Test player falls without ground support - GREEN: Constant downward acceleration - VERIFY: Physics simulation matches expected trajectory - Acceptance Criteria: - ☐ Gravity constant throughout - ☐ Terminal velocity enforced - ☐ Ground detection accurate - ☐ **Edge Case:** Falling through floor prevented

Day 6: Physics System Integration

Task 6.1: Cannon-es test integration - RED: Test physics world creation fails - GREEN: Initialize Cannon-es world in `Physics.ts` - VERIFY: Physics steps without errors - Acceptance Criteria: - ☐ Physics world created successfully - ☐ Physics step function works - ☐ Gravity configured correctly - ☐ **Mocking:** Physics can be stubbed for unit tests

Task 6.2: Body synchronization - RED: Test that physics body updates visual mesh - GREEN: Mesh-Body synchronization system - VERIFY: Physics simulation moves visual objects - Acceptance Criteria: - ☐ Visual mesh follows physics body - ☐ Synchronization happens every frame - ☐ No visual lag or jitter

Task 6.3: Collision detection tests - RED: Test collision events trigger game logic - GREEN: Cannon-es collision event handlers - VERIFY: Collisions trigger score changes or damage - Acceptance Criteria: - ☐ Collision events fire correctly - ☐ Collision types identified (player-obstacle, etc.) - ☐ Game logic responds appropriately - ☐ **Edge Case:** Multiple collisions in one frame handled

Day 6.5: Audio System (MOVED FROM MISSING)

Task 6.5.1: Audio system creation - RED: Test audio system creation fails - GREEN: Implement audio loading, playing, mixing in `Audio.ts` - VERIFY: All sounds play correctly - Acceptance Criteria: - ☐ AudioContext initialized - ☐ Background music loops correctly - ☐ Sound effects play on trigger - ☐ Volume controls work (master, music, SFX) - ☐ **Error Handling:** Audio load failure doesn't crash game

Task 6.5.2: Audio synchronization - RED: Test audio syncs with game events - GREEN: Audio triggers from game logic - VERIFY: Jump sound plays on jump, collision sound on hit - Acceptance Criteria: - ☐ Audio events triggered correctly - ☐ No audio lag or desync - ☐ Audio stops when game stops

Day 7: Scoring & Game Systems

Task 7.1: Score manager (pure functions) - RED: Test score increments/decrements - GREEN: Score manager with add/subtract methods - VERIFY: Score calculations are deterministic - Acceptance Criteria: - ☐ Score increases correctly - ☐ Score multipliers work - ☐ High score persists (LocalStorage) - ☐ **Edge Case:** Score overflow handled

Task 7.2: Game state machine - RED: Test state transitions (menu → playing → gameOver) - GREEN: Finite state machine implementation - VERIFY: Only valid transitions allowed - Acceptance Criteria: - ☐ All state transitions defined - ☐ Invalid transitions blocked - ☐ State change events emitted

Task 7.3: Power-up system - RED: Test power-up collection triggers effects - GREEN: Power-up entity with collection handlers - VERIFY: Power-ups disappear, effects apply - Acceptance Criteria: - ☐ All 4 power-up types work - ☐ Effects duration correct - ☐ Power-up visual feedback - ☐ **Edge Case:** Multiple power-ups collected simultaneously

WEEK 3: UI, INTEGRATION & POLISH

Day 8: HTML UI (DOM-based, No CORS!)

Task 8.1: DOM-based UI components - RED: Test UI elements render and update - GREEN: HTML overlay system in `systems/UI.ts` - VERIFY: Score displays update in real browser - Acceptance Criteria: - ☐ All UI elements render - ☐ Score updates in real-time - ☐ UI responsive to window resize - ☐
Edge Case: UI scales correctly on resize

Task 8.2: Menu navigation flow - RED: Test START button transitions scene - GREEN: Scene transition system with UI events - VERIFY: Click start → game starts - Acceptance Criteria: - ☐ Main menu displays correctly - ☐ Start button works - ☐ Settings accessible - ☐ **Accessibility:** Keyboard navigation works

Day 9: Integration Testing

Integration test suite: 1. Game start to gameplay flow 2. Player movement with obstacle avoidance 3. Score collection and display 4. Game over and restart cycle

Automated browser tests with Playwright: - Critical path automation - Visual regression testing - Performance validation

Acceptance Criteria: - ☐ All 4 critical paths tested - ☐ E2E tests pass consistently - ☐ Visual regression detected

Day 10: Optimization & Polish

Performance testing: - Frame rate under load testing (100+ obstacles) - Memory leak detection (5+ minute gameplay) - Asset loading optimization

Cross-browser testing: - Chrome (latest 3 versions) - Firefox (latest 3 versions) - Safari (latest 2 versions) - Mobile web validation

Acceptance Criteria: - ☐ 60 FPS maintained with 100+ objects - ☐ No memory leaks after 5 minutes - ☐ All browsers pass basic functionality - ☐ Input latency < 100ms

WEEK 4: REAL BROWSER TESTING & VALIDATION

Day 11: Real Browser E2E Testing

Task 11.1: Playwright E2E setup - RED: Test that Playwright fails (not installed) - GREEN: Install Playwright, configure test suite - VERIFY: E2E tests run in real Chrome/Firefox - Acceptance Criteria: - ☐ Playwright installed and configured - ☐ Tests run in real browsers (not headless for debugging) - ☐ Screenshots captured on failure

Task 11.2: Critical path E2E tests - RED: Test that game starts from menu - GREEN: E2E test: Menu → Start → Play → Game Over - VERIFY: Full game flow works in real browser - Acceptance Criteria: - ☐ Menu renders correctly - ☐ Start button clickable - ☐ Game loads and accepts input - ☐ Game over screen appears - ☐ Restart works

Task 11.3: Visual regression testing - RED: Test that visual changes are detected - GREEN: Baseline screenshots + comparison - VERIFY: Unexpected visual changes flagged - Acceptance Criteria: - ☐ Baseline screenshots captured - ☐ Changes detected on regression - ☐ False positives minimized

Day 12: Accessibility & Edge Cases

Task 12.1: Accessibility testing - RED: Test that keyboard-only navigation fails - GREEN: Implement keyboard navigation for all UI - VERIFY: Screen reader announces UI elements - Acceptance Criteria: - ☐ All UI keyboard accessible - ☐ Focus indicators visible - ☐ ARIA labels present - ☐ Color contrast meets WCAG AA

Task 12.2: Edge case testing - RED: Test game breaks on edge cases - GREEN: Add tests for edge cases: - Window resize during gameplay - Tab switch (blur/focus) - Network disconnect (if applicable) - Low memory conditions - VERIFY: Game handles gracefully - Acceptance Criteria: - ☐ Resize handled without crash - ☐ Focus/blur handled correctly - ☐ Error messages user-friendly

Day 13: Documentation & Handoff

Task 13.1: Test documentation - RED: Test that test documentation is missing - GREEN: Create `TESTING.md` with: - How to run tests - Test structure overview - Coverage report interpretation - Common test patterns - VERIFY: New developer can run tests - Acceptance Criteria: - ☐ Testing documentation complete - ☐ Examples provided - ☐ Troubleshooting section included

Task 13.2: Code coverage report - RED: Test that coverage report shows <90% - GREEN: Achieve 90%+ coverage across all systems - VERIFY: Coverage report meets targets - Acceptance Criteria: - ☐ Overall coverage ≥90% - ☐ Critical paths 100% - ☐ Coverage report generated

Day 14: Final Validation & Buffer

Task 14.1: Full regression test - Run entire test suite - Fix any failures - Verify 90%+ coverage

Task 14.2: Performance benchmark - Run performance tests - Verify 60 FPS target - Document baseline metrics

Task 14.3: Cross-browser final pass - Test on Chrome, Firefox, Safari - Fix any browser-specific issues

MISSING SYSTEMS TO IMPLEMENT (TDD)

Critical Files (Create with TDD)

1. `src/systems/Audio.ts` - Audio management system
2. RED: Test audio system creation fails
3. GREEN: Implement audio loading, playing, mixing
4. VERIFY: All sounds play correctly
5. `src/systems/UI.ts` - UI management system
6. RED: Test UI system creation fails
7. GREEN: Implement DOM-based UI components
8. VERIFY: All UI elements functional

9. `src/scenes/MenuScene.ts` - Main menu scene
 10. RED: Test menu scene fails
 11. GREEN: Implement menu with navigation
 12. VERIFY: Menu transitions work
 13. `src/scenes/GameOverScene.ts` - Game over screen
 14. RED: Test game over scene fails
 15. GREEN: Implement game over display
 16. VERIFY: Score display and restart work
-

TEST COVERAGE TARGETS

Phase Targets:

- **Week 0:** 20%+ coverage (baseline established)
- **Week 1:** 50%+ coverage (core systems)
- **Week 2:** 70%+ coverage (gameplay systems)
- **Week 3:** 85%+ coverage (integration)
- **Week 4:** 90%+ coverage (complete game)

Critical Paths (100% coverage required):

1. Game initialization and cleanup
 2. Input system → player movement
 3. Collision detection → game logic
 4. State transitions (menu → play → gameOver)
 5. Score calculation and persistence
 6. Power-up collection and effects
-

PERFORMANCE BENCHMARKS (Testable)

Technical Metrics:

- **Frame Rate:** ≥60 FPS with 100+ dynamic objects
- **Frame Time:** <16ms for 60 FPS, <33ms for 30 FPS

- **Memory:** <200MB peak usage
- **Load Time:** <3s on 4G, <1s local
- **Input Latency:** <100ms
- **Object Budget:** Support 100+ dynamic objects at 60 FPS

Performance Test Suite:

```
// Example performance test
it('should maintain 60 FPS with 100 obstacles', async () => {
  const fps = await measureFPS(1000bstacles);
  expect(fps).toBeGreaterThanOrEqual(60);
});

it('should not leak memory over 5 minutes', async () => {
  const initialMemory = getMemoryUsage();
  await runGameLoop(5 * 60); // 5 minutes
  const finalMemory = getMemoryUsage();
  expect(finalMemory - initialMemory).toBeLessThan(10); // <10MB incre
});
```

CROSS-BROWSER REQUIREMENTS

Supported Browsers:

- **Chrome:** Latest 3 versions
- **Firefox:** Latest 3 versions
- **Safari:** Latest 2 versions
- **Edge:** Latest 2 versions (Chromium-based)

WebGL Requirements:

- WebGL 1.0 minimum support
- WebGL 2.0 preferred for better performance
- Fallback strategy for unsupported browsers

Test Matrix:

Feature	Chrome	Firefox	Safari	Edge
Rendering	✓	✓	✓	✓
Physics	✓	✓	✓	✓
Input	✓	✓	✓	✓
Audio	✓	✓	✓	✓
UI	✓	✓	✓	✓

ACCESSIBILITY FEATURES

Required:

- Keyboard-only navigation support
- Color-blind friendly color schemes
- Adjustable text sizes
- Audio volume controls (music, SFX, master)
- Visual feedback for all audio events
- Screen reader compatibility for UI elements

Accessibility Tests:

- [] All UI elements keyboard accessible
 - [] Focus indicators visible
 - [] ARIA labels present
 - [] Color contrast meets WCAG AA
 - [] Screen reader announces key events
-

ASSET PIPELINE

Asset Types:

- **Textures:** PNG/JPG with mipmaps
- **Models:** GLTF/GLB format
- **Audio:** OGG (preferred), MP3 fallback
- **Fonts:** WOFF2 format

Loading Strategy:

- Progressive loading with progress indicator
- Asset caching for repeated access
- Fallback assets for missing resources
- Async loading with error handling

Asset Test Requirements:

- ☐ All assets load successfully
 - ☐ Loading progress displays correctly
 - ☐ Fallback assets work when primary fails
 - ☐ No blocking on asset load
-

PERSISTENCE & SAVE SYSTEM

Data to Persist:

- High scores (top 10)
- User settings (audio, controls, graphics)
- Game progress (for pause/resume)

Implementation:

- LocalStorage for browser-based persistence
- Data encryption for sensitive info
- Backup/restore functionality

Test Requirements:

- ☐ High scores persist across sessions
- ☐ Settings saved and loaded correctly
- ☐ Game state can be saved/restored
- ☐ Data integrity maintained

VERIFICATION AT EACH COMMIT

Before any commit: 1. Run unit tests: `npm run test` 2. Run integration tests: `npm run test:integration` 3. Generate coverage report: `npm run test:coverage` 4. Manual browser test of changed feature 5. Verify no console errors or warnings 6. Confirm code follows project style guide

Coverage Requirements: - Overall coverage $\geq 90\%$ - Critical paths 100% - No uncovered new code

DELIVERABLE VALIDATION CHECKLIST

Before marking any task as complete: - ☐ Tests written first (RED phase verified) - ☐ Implementation passes tests (GREEN verified) - ☐ All existing tests still pass (no regression) - ☐ Coverage report shows adequate coverage - ☐ Manual browser test confirms functionality - ☐ No console errors or warnings - ☐ Code follows project style guide - ☐ Performance benchmarks met - ☐ Cross-browser testing completed - ☐ Accessibility requirements met

Only when ALL checks pass → Task is DONE.

RISK REGISTER & MITIGATION

Risk	Probability	Impact	Mitigation	Test Coverage
Three.js complexity	Medium	High	Start small, test visualization early	Unit tests for scene/renderer

Risk	Probability	Impact	Mitigation	Test Coverage
Physics integration	High	High	Mock physics, integrate gradually	Integration tests for Cannon-es
Browser compatibility	Medium	Medium	Early cross-browser testing	E2E tests on multiple browsers
Performance issues	Medium	High	Profile early, optimize systematically	Performance test suite
Memory leaks	Medium	High	Regular memory profiling	Memory leak detection tests
Audio sync issues	Low	Medium	Test audio timing early	Audio synchronization tests
Existing code conflicts	High	High	Retroactive TDD, don't delete	Baseline tests before refactor

SUCCESS METRICS (Testable)

Technical Metrics:

- 90%+ test coverage (branch)
- 60 FPS maintained under load
- <3 second initial load time
- No memory leaks in 5+ minute gameplay
- Input latency < 100ms
- All UI elements functional cross-browser

User Experience Metrics:

- Intuitive controls (WASD + SPACE)
 - No visual artifacts or glitches
 - Smooth animations (60 FPS)
 - Responsive UI feedback
 - Clear visual/audio feedback for all actions
-

NEXT ACTION: START FROM WEEK 0

Current State: Code exists without TDD. Will use retroactive TDD approach.

Immediate Steps: 1. **Week 0 Infrastructure:** Setup testing infrastructure (Vitest, mocking, CI) 2. **Baseline Tests:** Write tests for existing code to establish baseline 3. **Start Week 1:** Begin strict TDD for new features 4. **Real Browser Testing:** Use Playwright for E2E validation (Week 4)

First Task: 1. Create `vite.config.ts` with Vitest configuration 2. Create `tests/mocks/` directory with Three.js/Cannon-es mocks 3. Create `.github/workflows/test.yml` for CI 4. Write first failing test for project structure

APPENDIX: COMPLETE TEST INVENTORY

Unit Tests Required (100+)

- Game class (start, pause, resume, stop)
- GameLoop timing and frame scheduling
- SceneManager (register, unregister, transitions)
- InputSystem (keyboard, mouse, gamepad)
- PhysicsSystem (world creation, body management, collision)
- Player (movement, jump, damage, health, score)
- Obstacle (creation, movement, collision types)
- PowerUp (creation, effects, duration)
- ScoreManager (calculation, persistence, multipliers)
- AssetLoader (loading, caching, error handling)
- AudioSystem (loading, playing, mixing, controls)

- UISystem (rendering, updates, interactions)
- Logger utilities
- Constants validation

Integration Tests Required (20+)

- Game start to gameplay flow
- Player movement with obstacle avoidance
- Score collection and display
- Game over and restart cycle
- Scene transitions (Boot → Menu → Game → GameOver)
- Physics-visual synchronization
- Audio-system integration
- UI-system integration
- Power-up collection and effects
- High score persistence

E2E Tests Required (10+)

- Full game loop from menu to game over
- Visual regression testing
- Performance validation (60 FPS)
- Cross-browser compatibility
- Accessibility validation
- Mobile touch input (if applicable)
- Audio functionality
- Settings persistence
- Real browser input testing
- Menu navigation flow

Performance Tests Required (5+)

- Frame rate under load
- Memory leak detection
- Asset loading optimization
- Input latency measurement
- Render thread vs physics thread separation

Total Tests Required: ~135+

TESTING THROUGH REAL BROWSER (CRITICAL FOR USER)

Why Real Browser Testing is Required: 1. Godot WebGL issues were only discoverable in real browser 2. CORS problems only manifest in browser environment 3. UI rendering bugs require actual DOM 4. Performance metrics need real hardware

Real Browser Test Setup:

```
# Install Playwright with browsers
npm install -D @playwright/test

# Run tests in real Chrome (not headless)
npx playwright test --headed

# Run specific test with debugging
npx playwright test --debug
```

E2E Test Example:

```
import { test, expect } from '@playwright/test';

test('game starts from menu', async ({ page }) => {
  await page.goto('/');

  // Menu should be visible
  await expect(page.locator('.menu-container')).toBeVisible();

  // Click start button
  await page.click('#start-btn');

  // Game should start
  await expect(page.locator('.hud-score')).toBeVisible();

  // Player should be on screen
```

```
const canvas = page.locator('canvas');  
await expect(canvas).toBeVisible();  
});
```

This plan supersedes all previous versions. Strict TDD enforcement required for all development. Updated: Retroactive TDD strategy, 4-week timeline, real browser E2E testing emphasis.